

Experiment No. 3

Experiment Name - Implementation and Analysis of Spatial Filtering Techniques using Low-Pass and High-Pass Filters

Steps

NAME : Atul Krishna
COURSE : B.Tech CSE
SECTION - B
SUBJECT : Computer Vision
ROLL NO. : 14
CU ID : CU24250231

1. Import the required libraries and load a grayscale or color image for processing.

```
# Importing the required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload image from your computer
uploaded = files.upload()

# Get the uploaded image name automatically
image_name = list(uploaded.keys())[0]

# Read the image
img = cv2.imread(image_name)

# Convert BGR image to RGB for Matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Image uploaded successfully!")
print("Image size:", img.shape)
```

my photo.jpg
my photo.jpg(image/jpeg) - 12084 bytes, last modified: 8/20/2026 - 100% done
Saving my photo.jpg to my photo (1).jpg
Image uploaded successfully!
Image size: (300, 217, 3)

2. Display the original image and analyze its visual characteristics.

```
# Displaying the original image

plt.figure(figsize=(4, 3))

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.show()

# Observation:
# The original image contains different intensity values, edges, textures, and fine details.
```

Original Image



3. Apply Gaussian Blur to reduce image noise while preserving important structures.

```
# Applying Gaussian Blur

gaussian_blur = cv2.GaussianBlur(img_rgb, (5, 5), 0)

plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gaussian_blur)
plt.title("Gaussian Blur")
plt.axis("off")

plt.show()

# Observation:
# Gaussian filtering smooths the image and reduces noise while preserving important structures better than simple averaging.
```

Original Image

Gaussian Blur



4. Apply Median Filtering to remove impulse (salt-and-pepper) noise from the image.

```
# Applying Median Filtering

median_blur = cv2.medianBlur(img_rgb, 5)

plt.figure(figsize=(4, 3))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(median_blur)
plt.title("Median Filter")
plt.axis("off")

plt.show()

# Observation:
# Median filtering removes impulse or salt-and-pepper noise effectively while preserving edges.
```

Original Image

Median Filter



5. Apply Average (Mean) Filtering and compare its smoothing performance with Gaussian filtering.

```
# Applying Average (Mean) Filtering

average_blur = cv2.blur(img_rgb, (4, 4))

plt.figure(figsize=(5, 4))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(average_blur)
plt.title("Average Filter")
plt.axis("off")
```

```
plt.subplot(1, 3, 3)
plt.imshow(gaussian_blur)
plt.title("Gaussian Filter")
plt.axis("off")
```

```
plt.show()
```

Observation:

Average filtering provides strong smoothing but may blur edges.

Gaussian filtering gives smoother and more natural results with better edge preservation.

Original

Average Filter

Gaussian Filter



6. Implement Laplacian Filtering to enhance image edges and fine details.

Applying Laplacian Filtering

```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
```

Convert values into displayable range

```
laplacian = cv2.convertScaleAbs(laplacian)
```

```
plt.figure(figsize=(6, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Edge Detection")
plt.axis("off")
```

```
plt.show()
```

Observation:

The Laplacian operator highlights rapid intensity changes and detects edges and fine details.

Original Grayscale Image

Laplacian Edge Detection



7. Apply Sobel Edge Detection in both horizontal and vertical directions to detect image gradients.

Applying Sobel Edge Detection

Detect changes in X direction

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

Detect changes in Y direction

```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

Convert results into displayable format

```

sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)

# Combine horizontal and vertical edges
sobel_combined = cv2.addWeighted(
    sobel_x, 0.5,
    sobel_y, 0.5,
    0
)

plt.figure(figsize=(5, 4))

plt.subplot(1, 3, 1)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")

plt.show()

# Observation:
# Sobel X detects intensity changes mainly in the horizontal x-direction.
# Sobel Y detects intensity changes mainly in the vertical y-direction.
# Combining both gives a more complete representation of image edges.

```

Sobel X



Sobel Y



Combined Sobel



8. Compare the outputs of different low-pass and high-pass filters in terms of noise reduction, edge preservation, and computational performance.

```

import time

# Average Filter
start = time.perf_counter()
average_blur = cv2.blur(img_rgb, (5, 5))
average_time = time.perf_counter() - start

# Gaussian Filter
start = time.perf_counter()
gaussian_blur = cv2.GaussianBlur(img_rgb, (5, 5), 0)
gaussian_time = time.perf_counter() - start

# Median Filter
start = time.perf_counter()
median_blur = cv2.medianBlur(img_rgb, 5)
median_time = time.perf_counter() - start

# Laplacian Filter
start = time.perf_counter()
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)
laplacian_time = time.perf_counter() - start

# Sobel Filter
start = time.perf_counter()
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
sobel_time = time.perf_counter() - start

print("Computational Performance:")
print("Average Filter :", round(average_time, 6), "seconds")
print("Gaussian Filter:", round(gaussian_time, 6), "seconds")
print("Median Filter :", round(median_time, 6), "seconds")
print("Laplacian Filter:", round(laplacian_time, 6), "seconds")
print("Sobel Filter   :", round(sobel_time, 6), "seconds")

```

```
# Comparison:
# Average filter provides simple smoothing but may blur edges.
# Gaussian filter gives smoother results while preserving edges better.
# Median filter is effective for removing salt-and-pepper noise.
# Laplacian and Sobel filters are useful for detecting edges and fine details.
```

```
Computational Performance:
Average Filter : 0.000689 seconds
Gaussian Filter: 0.000656 seconds
Median Filter : 0.002283 seconds
Laplacian Filter: 0.001494 seconds
Sobel Filter : 0.00101 seconds
```

9. Display the original and filtered images along with suitable observations highlighting the strengths and limitations of each filtering technique.

```
plt.figure(figsize=(5, 4))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(average_blur)
plt.title("Average Filter")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(gaussian_blur)
plt.title("Gaussian Filter")
plt.axis("off")

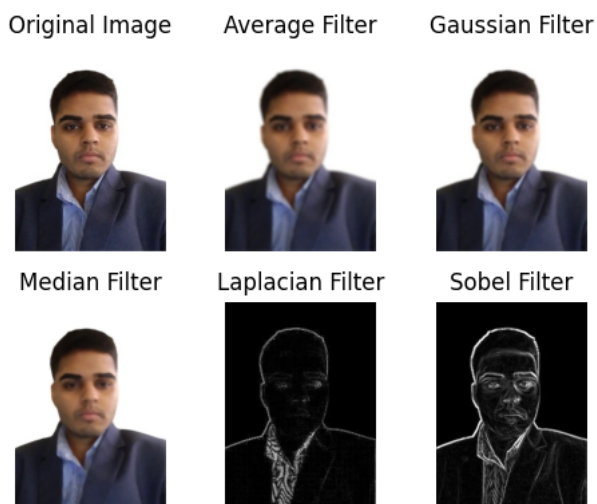
plt.subplot(2, 3, 4)
plt.imshow(median_blur)
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Filter")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Sobel Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

# Observations:
# 1. Average filter reduces noise but produces more edge blurring.
# 2. Gaussian filter provides smooth and natural noise reduction.
# 3. Median filter preserves edges better and is useful for impulse noise.
# 4. Laplacian filter highlights edges and fine image details.
# 5. Sobel filter detects horizontal and vertical edges effectively.
```



10. Interpret the results and discuss the applications of spatial filtering in real-world computer vision systems.

Interpretation of Results and Real-World Applications

The spatial filtering techniques show different effects on an image. The **Average Filter** smooths the image and reduces noise, but it may blur important edges. The **Gaussian Filter** provides smoother and more natural noise reduction while preserving edges better than the average filter. The **Median Filter** is highly effective for removing salt-and-pepper noise while maintaining important edges.

The **Laplacian Filter** enhances fine details and detects edges in different directions. The **Sobel Filter** detects horizontal and vertical edges and is useful for identifying object boundaries and shapes.

Spatial filtering techniques are widely used in **medical image processing** for noise reduction and detection of important structures, in **surveillance systems** for object and edge detection, in **remote sensing** for improving satellite images, and in **autonomous systems** for detecting objects, boundaries, and road features.

Thus, low-pass filters are mainly useful for **smoothing and noise reduction**, whereas high-pass filters are useful for **edge detection and detail enhancement**.

Questions and Answers

Q1. What is spatial filtering? How is it used in digital image processing?

Answer: Spatial filtering is a digital image processing technique in which the value of a pixel is modified using the values of its neighboring pixels. It is performed directly on the pixels of an image using a small matrix called a **filter or kernel**.

Spatial filtering is mainly used for:

- Removing noise from images.
- Smoothing and blurring images.
- Detecting edges and boundaries.
- Enhancing fine details.
- Improving image quality before further processing.

For example, Gaussian and Median filters are used for smoothing, while Sobel and Laplacian filters are used for edge detection.

Q2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Answer:

Low-Pass Filter	High-Pass Filter
Allows slowly changing intensity regions to remain.	Emphasizes rapidly changing intensity regions.
Reduces high-frequency components such as noise and fine details.	Enhances high-frequency components such as edges and fine details.
Produces a smoother image.	Produces an image with sharper edges.
Mainly used for noise reduction and smoothing.	Mainly used for edge detection and sharpening.
Examples: Average, Gaussian, Median filters.	Examples: Sobel, Laplacian filters.

Thus, **low-pass filters smooth an image**, whereas **high-pass filters highlight important details and edges**.

Q3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Answer:

Filter	Working Principle	Main Application
Average Filter	Replaces each pixel with the average of neighboring pixel values.	General smoothing and noise reduction.
Gaussian Filter	Uses a Gaussian-weighted kernel where nearby pixels have more importance.	Smooth noise while maintaining better edge quality.
Median Filter	Replaces a pixel with the median value of its neighborhood.	Removing salt-and-pepper noise.

The **Average Filter** is simple but can blur edges. The **Gaussian Filter** produces smoother and more natural results. The **Median Filter** is particularly useful when the image contains impulse or salt-and-pepper noise.

Q4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Answer: Salt-and-pepper noise appears as random **black and white pixels** in an image. The Median Filter replaces the central pixel with the median value of its neighboring pixels.

Unlike an average filter, the median filter does not allow extreme values to strongly affect the result. Therefore, isolated black or white pixels are removed while the important edges of the image are preserved.

Hence, the Median Filter is highly effective for removing **salt-and-pepper or impulse noise**.

Q5. Explain the role of convolution kernels in spatial filtering.

Answer: A convolution kernel is a small matrix of numerical values used to process pixels and their neighboring pixels. The kernel is moved across the image, and mathematical operations are performed between the kernel values and the corresponding image pixels.

Different kernels produce different effects.

For example:

- **Average kernel** → image smoothing.
- **Gaussian kernel** → smooth noise with weighted averaging.
- **Sobel kernel** → edge detection.

- **Laplacian kernel** → enhancement and detection of fine edges.

Therefore, the convolution kernel determines the type and strength of spatial filtering applied to an image.

Q6. What is the purpose of the Sobel and Laplacian operators in edge detection?

Answer: The **Sobel operator** is used to detect edges by calculating changes in pixel intensity in the horizontal and vertical directions. It uses two kernels, commonly called **Sobel-X** and **Sobel-Y**.

The **Laplacian operator** calculates the second-order derivative of image intensity and detects regions where the intensity changes rapidly. It can detect edges in multiple directions.

Therefore:

- **Sobel** → detects horizontal and vertical edges and provides directional information.
- **Laplacian** → detects edges and fine details in multiple directions.

Both are widely used for edge detection and feature extraction.

Q7. Why are filtering operations considered an essential preprocessing step in computer vision?

Answer: Filtering is an important preprocessing step because real-world images often contain noise, unwanted variations, and unnecessary details. Filtering improves the quality of an image before it is given to a computer vision algorithm.

It helps to:

- Remove unwanted noise.
- Smooth the image.
- Improve important features.
- Enhance edges and boundaries.
- Make object detection and segmentation more reliable.

For example, applying a Gaussian or Median Filter before object detection can reduce noise and improve the performance of the vision system.

Q8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Answer: Image smoothing reduces noise and unwanted variations, but excessive smoothing can also remove important edges and fine details.

For example, an **Average Filter** can effectively smooth an image but may blur object boundaries. A **Gaussian Filter** provides better controlled smoothing, while a **Median Filter** can remove impulse noise while preserving edges relatively well.

Therefore, there is a trade-off:

More smoothing → less noise but more loss of edges and details.

Less smoothing → better edge preservation but more noise may remain.

The appropriate filter and kernel size should be selected according to the application.

Q9. Mention four real-world applications where spatial filtering techniques are widely used.

Answer: Four major real-world applications of spatial filtering are:

1. **Medical Imaging** – Used for noise reduction and enhancement of X-ray, MRI, CT, and ultrasound images.
2. **Surveillance Systems** – Used for improving images and detecting object boundaries and movements.
3. **Remote Sensing** – Used to enhance satellite and aerial images for land and environmental analysis.
4. **Autonomous Vehicles** – Used for detecting road boundaries, objects, obstacles, and other important features.

Spatial filtering is also used in robotics, industrial inspection, facial recognition, and image enhancement.

Q10. Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Answer:

Spatial Domain Filtering	Frequency Domain Filtering
Works directly on image pixels.	Works on the frequency representation of an image.
Uses kernels or masks for filtering.	Uses transformations such as Fourier Transform.
Relatively simple to implement.	Usually requires transformation to and from the frequency domain.
Suitable for local image operations.	Useful for analyzing and controlling different frequency components.
Examples: Average, Gaussian, Median, Sobel, Laplacian.	Examples: Frequency-domain low-pass and high-pass filters.
Commonly used for smoothing, noise removal, and edge detection.	Commonly used for periodic noise removal, image enhancement, and frequency-based analysis.

In conclusion, spatial domain filtering is generally simpler and works directly on pixels, making it suitable for many practical image-processing tasks. Frequency domain filtering provides better control over the frequency components of an image and is useful for applications involving periodic noise and frequency-based image enhancement.